

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO TUẦN 7
MÔN THỰC TẬP CƠ SỞ

**Đề tài: Nghiên cứu, đánh giá và triển khai
mô hình Deep Learning tối ưu cho hệ thống phân loại rác thải**

Giảng viên hướng dẫn:	TS. Kim Ngọc Bách
Sinh viên thực hiện:	Nguyễn Văn Gia Bảo
Mã sinh viên:	B23DCCN079

Hà Nội - 2026

MỤC LỤC

1. Mục tiêu công việc trong tuần	3
2. Nội dung công việc đã thực hiện	3
2.1 Lựa chọn và thiết kế cấu trúc Cơ sở dữ liệu (Database)	3
2.2 Xây dựng module Thống kê trực quan (Dashboard) với Pandas và Plotly	4
2.3 Nghiên cứu vòng đời ứng dụng và tối ưu hóa hiệu năng bằng Session State	4
3. Kết quả đạt được	5
4. Khó khăn gặp phải	6
5. Kế hoạch tuần tiếp theo	6

1. Mục tiêu công việc trong tuần

Sau khi hoàn thiện mô hình nhận diện 10 lớp rác thải ở các tuần trước, hệ thống web hiện tại đang dừng ở mức "Stateless" (không lưu giữ trạng thái) – tức là quét xong ảnh, trả kết quả rồi xóa bỏ thông tin. Tuy nhiên, để đáp ứng yêu cầu của một phần mềm thực tế, hệ thống cần phải có khả năng lưu vết (log), quản lý lịch sử và báo cáo phân tích. Do đó, mục tiêu của tuần 7 được đặt ra bao gồm:

- Nghiên cứu, so sánh các giải pháp lưu trữ dữ liệu cục bộ và triển khai tích hợp Cơ sở dữ liệu để ghi nhận toàn bộ phiên làm việc của người dùng.
- Ứng dụng các thư viện phân tích dữ liệu (Data Analysis) để trích xuất, xử lý số liệu thô từ CSDL thành các chỉ số (metrics) mang tính quản trị.
- Xây dựng Bảng điều khiển (Dashboard) tương tác trực quan.
- Giải quyết bài toán giật lag và hao tốn tài nguyên phần cứng (CPU/GPU) do cơ chế tự động render lại trang của framework Streamlit gây ra.

2. Nội dung công việc đã thực hiện

2.1 Lựa chọn và thiết kế cấu trúc Cơ sở dữ liệu (Database)

Để hệ thống có thể lưu vết (log) lịch sử phân loại, em cần tích hợp một Cơ sở dữ liệu vào ứng dụng. Sau khi tìm hiểu, em quyết định không sử dụng các Hệ quản trị CSDL lớn (như MySQL hay SQL Server) vì chúng đòi hỏi cấu hình môi trường máy chủ phức tạp. Thay vào đó, em lựa chọn **SQLite**.

- **Lý do chọn SQLite:** Đây là một CSDL quan hệ siêu nhẹ, được tích hợp sẵn trong thư viện chuẩn của Python (sqlite3). Nó lưu trữ toàn bộ dữ liệu chỉ trong một file duy nhất (trash_history.db) nằm ngay tại thư mục chứa mã nguồn, giúp việc khởi tạo và quản lý cực kỳ thuận tiện, không yêu cầu cài đặt thêm phần mềm bên thứ ba.
- **Cấu trúc CSDL (Schema):** Em tiến hành thiết kế bảng history để lưu trữ thông tin mỗi phiên quét ảnh, bao gồm các trường:
 - id (Khóa chính, Tự tăng): Dùng để định danh duy nhất cho mỗi lượt quét.
 - scan_time (TEXT): Lưu lại thời gian thực (ngày, giờ) lúc hệ thống thực hiện phân tích ảnh.
 - trash_type (TEXT): Tên chi tiết của loại rác mà AI nhận diện được (VD: battery, cardboard...).
 - confidence (REAL): Mức độ tự tin của AI (tính theo %), giúp em đánh giá được hiệu năng thực tế của mô hình khi gặp các ảnh bên ngoài.

- category (TEXT): Nhóm rác lớn (Tái chế, Hữu cơ, Vô cơ, Nguy hại). Thay vì dùng lệnh truy vấn gom nhóm phức tạp lúc vẽ biểu đồ, em viết logic xử lý ở code Python để ánh xạ trash_type sang category và lưu sẵn vào cột này, giúp tối ưu tốc độ trích xuất dữ liệu sau này.

2.2 Xây dựng module Thông kê trực quan (Dashboard) với Pandas và Plotly

Để ứng dụng mang lại giá trị quản trị hệ thống, em tạo thêm Tab "**Lịch sử & Thống kê**". Logic xử lý bao gồm 3 phân hệ chính:

- **Xử lý ngoại lệ (Exception Handling):** Kiểm tra trạng thái Database, nếu CSDL rỗng (người dùng chưa quét ảnh nào), hệ thống sẽ hiển thị thông báo hướng dẫn thay vì vắng lỗi chương trình.
- **Khai phá dữ liệu với Pandas:** Em sử dụng hàm `pd.read_sql_query()` để chuyển đổi toàn bộ truy vấn SQL thành cấu trúc DataFrame. Nhờ sức mạnh của Pandas, em tính toán các chỉ số (KPIs) hệ thống chỉ với một vài hàm tích hợp sẵn như `df['confidence'].mean()` (Tính % chính xác trung bình) hay `df['trash_type'].mode()` (Tìm loại rác có tần suất xả thải cao nhất).
- **Trực quan hóa với Plotly:** Thay vì dùng Matplotlib sinh ra các bức ảnh tĩnh nhàm chán, em đã học cách dùng `plotly.express` để vẽ các biểu đồ động (Interactive Charts). Cụ thể là Biểu đồ Donut Chart (Cơ cấu nhóm rác) và Bar Chart (Tần suất từng loại rác) cho phép người dùng di chuột (hover) để xem chi tiết số liệu.

2.3 Nghiên cứu vòng đời ứng dụng và tối ưu hóa hiệu năng bằng Session State

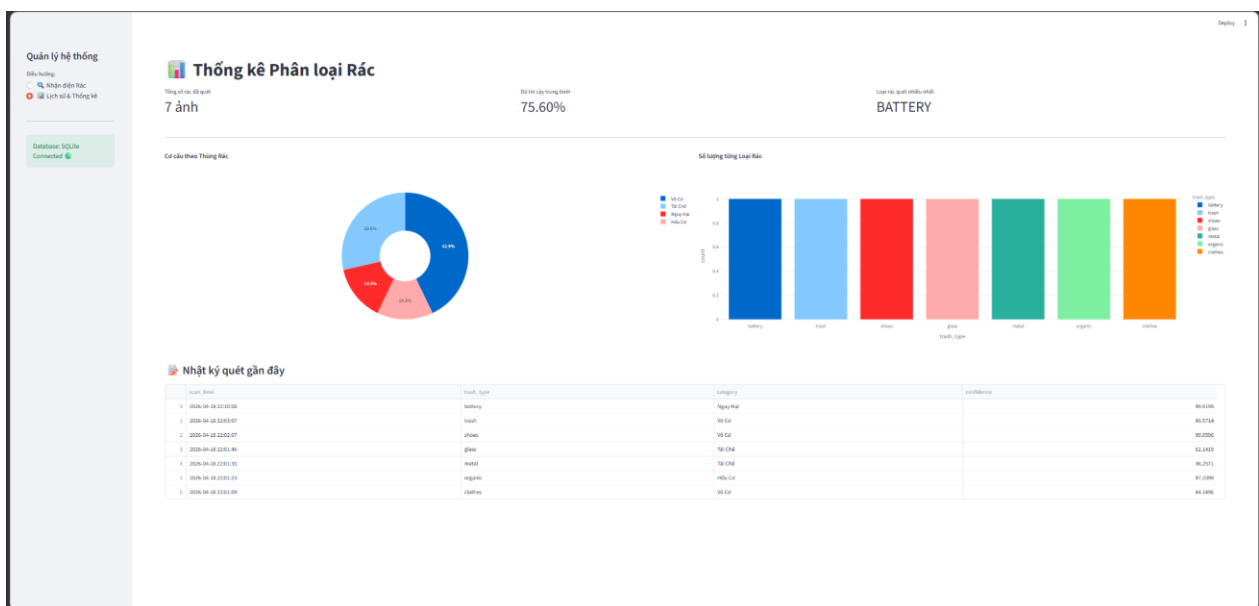
Đây là bài toán kỹ thuật phức tạp nhất trong tuần. Đặc thù kiến trúc của Streamlit là mô hình thực thi từ trên xuống dưới (Top-down Execution Model). Mỗi thao tác click chuột của người dùng đều khiến toàn bộ mã nguồn file `app.py` chạy lại từ đầu.

- **Phân tích vấn đề:** Do luồng chạy này, khi người dùng tải ảnh lên ở Tab "Nhận diện", AI sẽ chạy mất khoảng 1-2 giây để ra kết quả. Tuy nhiên, khi họ bấm sang Tab "Thống kê" rồi quay lại, biến lưu trữ ảnh bị reset thành rỗng (None). Nếu lưu ảnh lại mà không kiểm soát, AI sẽ bị kích hoạt chạy lại (Inference) trên chính bức ảnh cũ, vừa làm ứng dụng giật lag, gây tốn RAM/GPU vô ích, vừa tạo ra các dòng log dữ liệu trùng lặp lưu vào CSDL.
- **Thiết kế giải pháp:** Em đã tìm hiểu sâu vào cơ chế **Session State** (Bộ nhớ đệm phiên làm việc) của Streamlit để xây dựng một thuật toán kiểm soát trạng thái hoàn chỉnh:

1. Khi có file upload, hệ thống kiểm tra tên file (uploaded_file.name). Nếu là file mới, lưu ảnh vào st.session_state['uploaded_image'] và bật cờ trạng thái need_analyze = True.
2. Bọc khối lệnh chứa mô hình AI bên trong điều kiện if need_analyze. Nếu đúng là ảnh mới, AI mới được phép xử lý tensor, trả ra kết quả dự đoán và ghi lệnh INSERT vào CSDL.
3. Sau khi ghi xong, ngay lập tức lưu các text kết quả (% tin cậy, phân loại rác, lời khuyên xử lý) vào Session State và tắt cờ need_analyze = False.
4. Ở các lần render tiếp theo (khi người dùng chuyển Tab qua lại), hệ thống chỉ việc bốc dữ liệu tĩnh từ RAM (Session) ra để hiển thị. Giao diện tải ngay lập tức với độ trễ (latency) gần như bằng 0.

3. Kết quả đạt được

- Hoàn thiện 100% mục tiêu biến hệ thống thành một Web Application hoàn chỉnh với 2 tính năng song song: Nhận diện thời gian thực (Real-time Inference) và Quản trị dữ liệu (Data Management).
- Toàn bộ hành vi của người dùng được lưu vết an toàn vào tệp trash_history.db, sẵn sàng cho việc trích xuất và backup dữ liệu.
- Giải quyết triệt để lỗi mất dữ liệu trên giao diện và lỗi sinh rác dữ liệu trong Database. Quan trọng hơn, nhờ thuật toán chặn gọi AI nhiều lần bằng Session State, hiệu năng của ứng dụng được tối ưu đáng kể, loại bỏ hoàn toàn hiện tượng nghẽn cổ chai (Bottleneck) khi chuyển đổi qua lại giữa các màn hình.



Hình 1 - Giao diện Dashboard được xây dựng bằng Plotly, trực quan hóa dữ liệu lấy từ SQLite.

4. Khó khăn gặp phải

- Khó khăn 1: Xung đột luồng (Thread-safety) trong SQLite:** Trong quá trình dev, hệ thống thỉnh thoảng báo lỗi *"SQLite objects created in a thread can only be used in that same thread"*. Em nhận ra Streamlit xử lý các sự kiện click nút ở các tiến trình (thread) độc lập, dẫn đến việc dùng chung 1 đối tượng connection toàn cục bị lỗi.
 - Cách khắc phục: Áp dụng pattern mở/đóng kết nối theo từng phiên dịch vụ. Trong mỗi hàm `init_db()` hay `save_to_db()`, em đều khai báo một `conn = sqlite3.connect()` độc lập và bắt buộc gọi `conn.close()` ở cuối hàm để giải phóng tài nguyên luồng an toàn.
- Khó khăn 2: Đồng bộ hóa kiểu dữ liệu giữa Python và SQL:** Khi tính toán độ tin cậy của AI, giá trị trả về là dạng Tensor Float có độ chính xác rất dài, khi chèn vào SQLite bị sai lệch định dạng hiển thị.
 - Cách khắc phục: Xử lý ép kiểu dữ liệu từ mức Backend. Cụ thể, em sử dụng lệnh `item() * 100` để gỡ bỏ lớp Tensor, sau đó dùng hàm `round(..., 2)` để làm tròn thành số thập phân có 2 chữ số rồi mới lưu trữ vào trường REAL của SQLite.

5. Kế hoạch tuần tiếp theo

- Triển khai nghiên cứu Hệ thống xác thực người dùng (User Authentication) nhằm nâng cấp ứng dụng lên mức cá nhân hóa (Personalization).

- Tích hợp các thư viện mã hóa mật khẩu (như bcrypt) và xây dựng giao diện Đăng nhập/Đăng ký.
- Điều chỉnh lại Lược đồ CSDL: Bổ sung thêm bảng Users và tạo khóa ngoại (Foreign Key) liên kết với bảng history để đảm bảo mỗi tài khoản chỉ xem được biểu đồ thống kê xả rác của cá nhân mình.